methods/computer/dynamic_dispatch.rb

```
class Computer
  def initialize(computer_id, data_source)
    @id = computer_id
    @data_source = data_source
  end

  def mouse
    component :mouse
  end

  def cpu
    component :cpu
  end

  def keyboard
    component :keyboard
  end

  def component(name)
    info = @data_source.send "get_#{name}_info", @id
    price = @data_source.send "get_#{name}_price", @id
    result = "#{name.capitalize}: #{info} ($#{price})"
    return "* #{result}" if price >= 100
    result
  end
end
```

A call to mouse is delegated to component, which in turn calls DS#get_mouse_info and DS#get_mouse_price. The call also writes the capitalized name of the component in the resulting string. You open an irb session and smoke-test the new Computer:

```
my_computer = Computer.new(42, DS.new)
my_computer.cpu   # => * Cpu: 2.16 Ghz ($220)
```

This new version of Computer is a step forward because it contains far fewer duplicated lines—but you still have to write dozens of similar methods. To avoid writing all those methods, you can turn to define_method.

Step 2: Generating Methods Dynamically

You and Bill refactor Computer to use *Dynamic Methods* (51), as shown in the following code.